

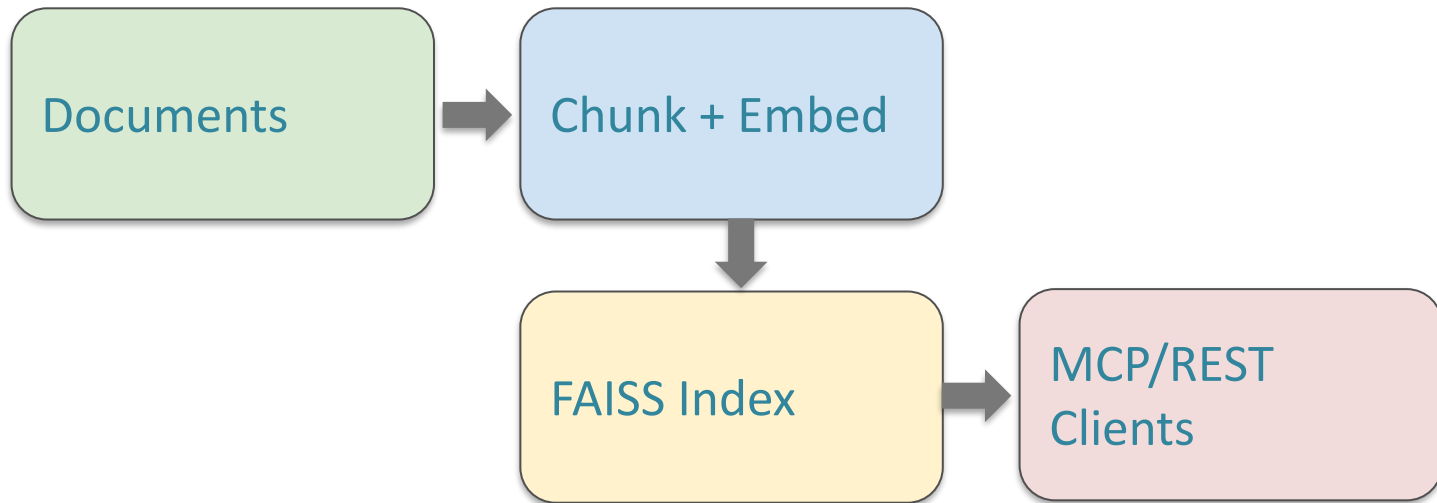
FAISS RAG Server: Local Retrieval + MCP Tooling for Team Workflows

- Private, local-first RAG service with FAISS indexing
- API + MCP endpoint for agent/tool ecosystems
- Focus today: embedding pipeline, MCP interoperability, and next upgrades

Problem being Solved

- Need grounded answers from docs/code without sending data externally
- Would like one retrieval service reusable across local apps and coding assistants
- Need a predictable interface for both REST and MCP tool-based workflows

Architecture

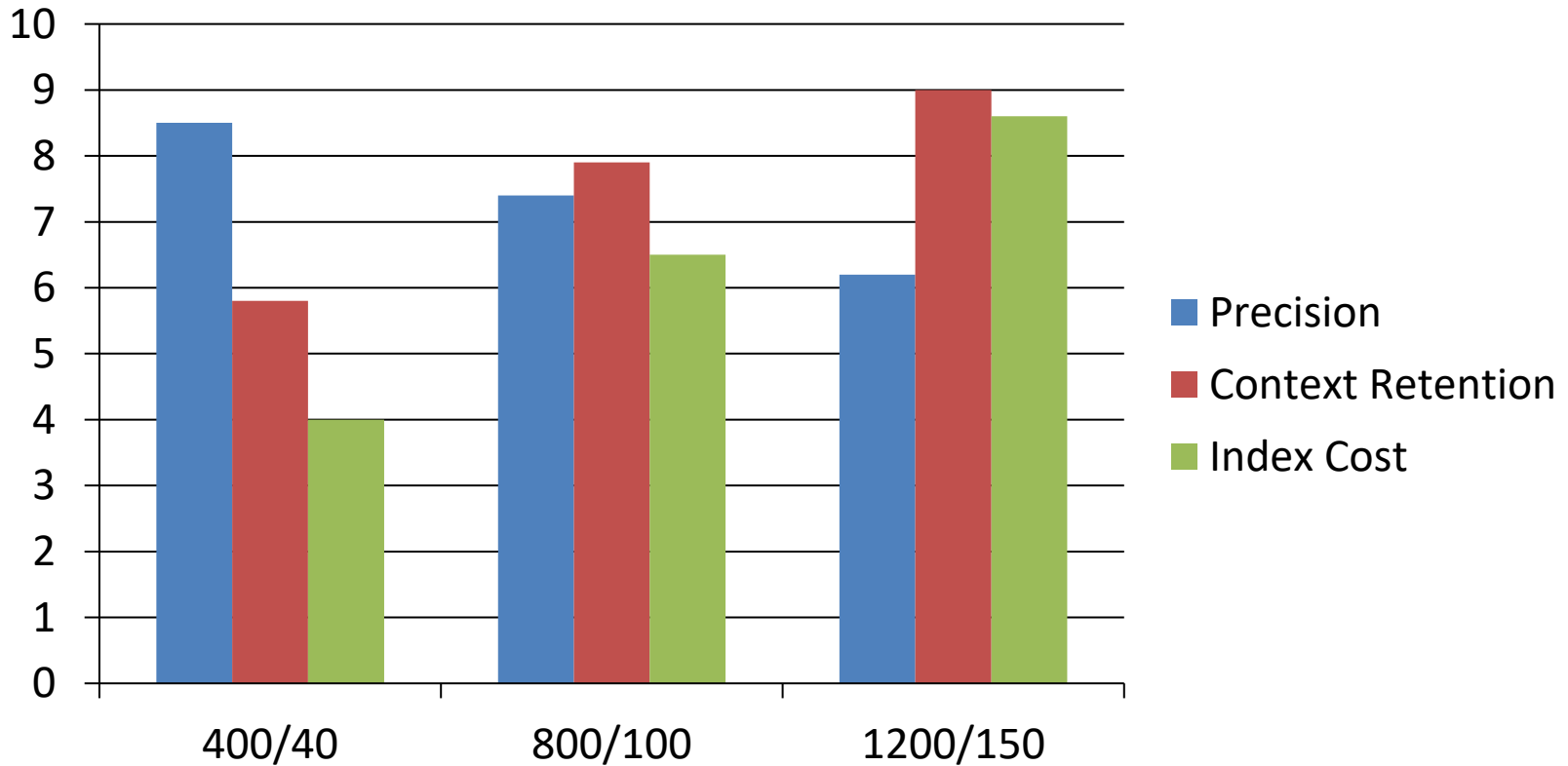


Single local retrieval layer serves both API and MCP clients with shared document grounding.

Current Design (Code + Runtime)

- FastAPI app exposes `/api/*` endpoints and `/mcp` JSON-RPC/SSE routes
- Documents are chunked, embedded, and stored in persistent FAISS + metadata files
- Retrieval path: query -> embedding -> FAISS search -> structured chunk payloads
- Storage survives restarts via `./documents` and `./vectorstore`

Chunk/Overlap Tradeoff



How to read this

- Smaller chunks trend toward precision
- Larger chunks retain more context
- Overlap increases cost and ingest time
- Use measured data for final tuning

Embedding & Adaptive Chunking

Embedding model: BAAI/bge-large-en-v1.5; defaults: CHUNK_SIZE=800, CHUNK_OVERLAP=100, TOP_K=3; index: IVF Flat (IndexIVFFlat).

Proposal: choose chunk size/overlap by source type instead of one global default

Document Type	Chunk Size	Overlap	Reason
API docs / design docs	900	120	Long sections; keep concept continuity
Code files	300	40	High precision around symbols/functions
Runbooks / SOP	650	100	Step boundaries need carry-over
Chat logs / tickets	450	60	Short units; avoid context bleed

Tuning approach: compare quality, latency, and index growth together with metadata-driven splitter presets and per-ingest overrides.

MCP Server & Standards Alignment

- MCP routes support initialization, tool listing, and tool execution
- Registered tools today: `retrieve_context` and `list_documents`
- Compatible transport pattern includes Streamable HTTP/SSE style interaction
- MCP home: <https://modelcontextprotocol.io>
- MCP specification:
<https://modelcontextprotocol.io/specification/2024-11-05>
- MCP inspector docs:
<https://modelcontextprotocol.io/docs/tools/inspector>
- JSON-RPC Error Codes:
https://www.jsonrpc.org/specification#error_object

Tool Compatibility Notes

(VS Code + Continue)

- Works as a general MCP tool provider for agent-capable clients including VS Code workflows
- Continue.dev interoperability note:
 - JSON content types are problematic in current flows
 - Current workaround returns JSON payloads serialized as type: text
 - This preserves schema-rich output while avoiding client parsing breakage
- Future improvement: client-specific adapters/content negotiation

Near-Term Improvements

- Add a re-ranker stage after FAISS retrieval to improve chunk ordering relevance
- Add a summarizer stage to compress retrieved context before prompt injection
- Combined effect: better precision, lower context-window pressure, more stable responses
- Proposed rollout:
 - Rollout phase 1: optional re-ranker behind config flag
 - Rollout phase 2: summarizer pass with token-budget guardrails
 - Rollout phase 3: quality metrics (grounding, citation usefulness, latency)

Forward Roadmap Discussion

- Expand to multimodal retrieval: text + image + audio metadata/transcripts
- Likely architecture: modality-specific ingestion + unified retrieval abstraction + fusion ranking
- Design concerns: storage growth, ingestion throughput, eval strategy, reindexing cost
- Discussion: prioritize chunk tuning/reranker first or begin multimodal ingestion now
- Discussion: define tier-1 client integrations (VS Code, Continue, OpenWeb UI, others)

CPU vs GPU indexing benchmark inside the container

Workload: 600 synthetic chunks, embedding model BAAI/bge-large-en-v1.5

- CPU total: ~405.56s
- GPU total (cuda:0 embeddings): ~87.97s
- Effective speedup: ~4.61x (embedding-dominated, so total and embedding speedups match)
 - CPU {'device': 'cpu', 'vectors': 600, 'dim': 1024, 'embed_s': 405.55755017499905, 'faiss_cpu_s': 0.003605698002502322, 'total_s': 405.56115587300155}
 - GPU {'device': 'cuda:0', 'vectors': 600, 'dim': 1024, 'embed_s': 87.96123269600139, 'faiss_cpu_s': 0.00838503200066043, 'total_s': 87.96961772800205}
 - speedup_embed_x 4.61
 - speedup_total_x 4.61

Workload: GPSMAP_8xxx_Series_OM_EN-US.pdf, 20.4 MB

(PyPDFLoader expanded to 1 doc per page – 278 docs)

- CPU total indexing time: 643.51s
- GPU total indexing time (RTX-A500 4GB 2048 cuda cores) (cuda:0 embeddings, FAISS CPU): 114.52s
- Effective Speedup: 5.62x (embed and total, since embedding dominates runtime)
 - CPU {'device': 'cpu', 'chunks': 952, 'dim': 1024, 'embed_s': 643.506423797, 'faiss_cpu_s': 0.008562470000470057, 'total_s': 643.5149862670005}
 - GPU {'device': 'cuda:0', 'chunks': 952, 'dim': 1024, 'embed_s': 114.49755090500184, 'faiss_cpu_s': 0.026597686002787668, 'total_s': 114.52414859100463}
 - speedup_embed_x 5.62
 - speedup_total_x 5.62